

Heart Disease MLOps Project

Reproducible model development, experiment tracking, API delivery and operational evidence

303

UCI Cleveland records

13

patient input features

2

cross-validated classifiers

Executive summary

This submission implements a complete heart-disease classification workflow: data acquisition and EDA, leakage-aware preprocessing, model comparison and MLflow tracking, a packaged scikit-learn pipeline, a tested FastAPI service, CI automation, Docker packaging, Kubernetes deployment manifests, request logging and Prometheus metrics.

The selected Logistic Regression model achieved mean five-fold cross-validation ROC-AUC **0.9082**. Its held-out ROC-AUC was **0.9632** on the fixed stratified split. Docker build, two-replica Docker Desktop Kubernetes rollout, live API checks and five local tests were executed on 2 October 2026; deployment evidence is included in this report.

Safety: This is an educational demonstration, not a medical device. It is not validated for diagnosis, triage or treatment and must not be used for clinical decisions.

Project: priyansh1114/MLOps · **Run environment:** Windows, Docker Desktop Linux engine · **Report date:** 2 October 2026

Dataset and data quality

The project uses the processed Cleveland subset of the UCI Heart Disease dataset.

`src/heart_disease_mlop/data_pipeline.py` downloads the UCI source file when the local snapshot is absent; the checked-in `data/heart.csv` makes routine runs independent of network access. Original target values 1–4 map to disease-present class 1; value 0 remains class 0.

303

rows

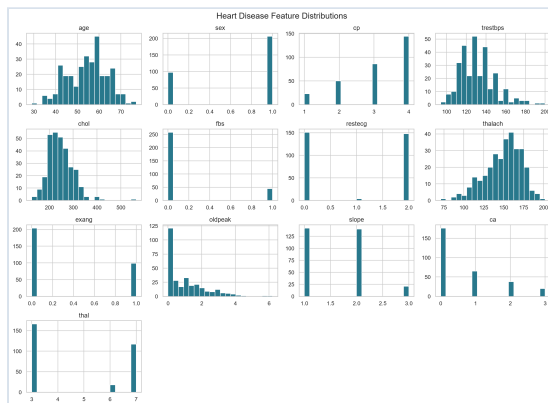
13

input features

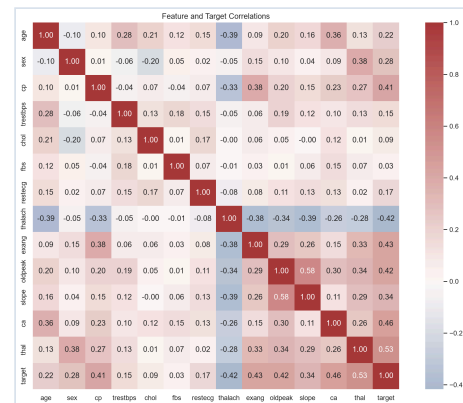
164 / 139

class 0 / class 1

Question marks are parsed as missing values. Four missing `ca` values and two missing `thal` values are preserved for training-only imputation. This avoids leakage from computing imputation statistics on the held-out partition. Feature histograms, feature/target correlations and the class balance chart are generated by the EDA command.



Feature distribution histograms



Feature and target correlation heatmap



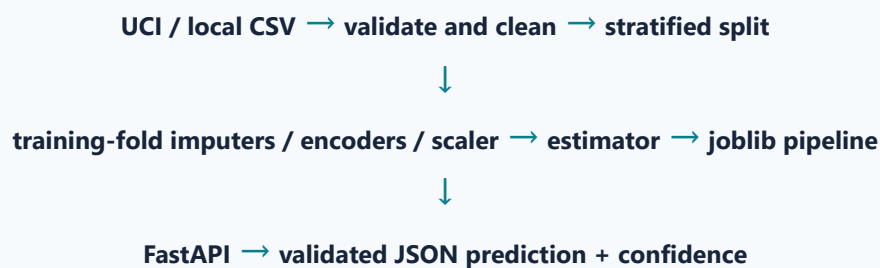
Target balance: 164 no-disease and 139 disease-present records

Generated EDA summary: `artifacts/eda/summary.json`. Source: [UCI processed Cleveland data](#).

One fitted pipeline from training to inference

The input schema is fixed in `FEATURE_COLUMNS`. Continuous-valued measurements use median imputation followed by standardization. Categorical attributes use most-frequent imputation followed by one-hot encoding with unknown-category tolerance. The complete fitted preprocessing-and-estimator pipeline is serialized together, so serving uses the same learned transformations as training.

Feature group	Columns	Training transformation
Continuous / measured	age, trestbps, chol, fbs, thalach, exang, oldpeak, ca	Median imputation; standard scaling
Coded categories	sex, cp, restecg, slope, thal	Most-frequent imputation; one-hot encoding
Target	target	0 = no disease; original 1–4 mapped to 1



Re-running the workflow

```
python -m pip install -r requirements.txt
python -m src.heart_disease_mlop.eda
python -m src.heart_disease_mlop.train_model
uvicorn api.main:app --host 127.0.0.1 --port 8000
```

`artifacts/best_model.joblib` holds the selected complete pipeline; per-model artifacts, evaluation tables and plots are also retained. The fixed random seed, stratified holdout and stratified five-fold CV make the experiment repeatable within the dependency environment.

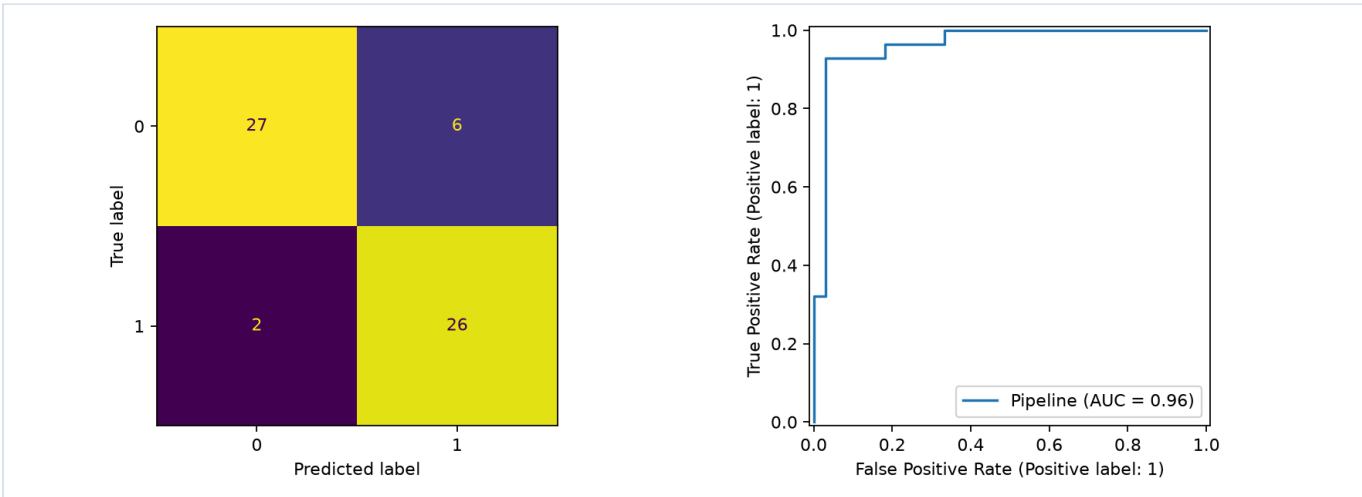
Artifact trust: joblib/pickle artifacts can execute code when loaded. Load only model files from this trusted project/build process. The dependency file uses minimum versions, not a fully resolved lock.

Two models, stratified five-fold CV

The dataset was split 80/20 with stratification and random seed 42. Grid search ran only on the training partition; mean CV ROC-AUC selected the final model. Accuracy, precision, recall and ROC-AUC were recorded for both CV and the independent holdout partition.

Model / best parameters	CV ROC-AUC mean ± SD	CV accuracy	CV precision	CV recall	Holdout accuracy	Holdout precision	Holdout recall	Holdout ROC-AUC
Logistic Regression C=0.1	0.9082 ± 0.0199	0.8386	0.8506	0.7921	0.8689	0.8125	0.9286	0.9632
Random Forest depth=None, leaf=2, trees=100	0.9001 ± 0.0217	0.8181	0.8186	0.7826	0.8689	0.8125	0.9286	0.9448

Logistic Regression is selected because it has the higher mean cross-validated ROC-AUC. The holdout metrics are descriptive for this small split; they are not used to choose hyperparameters. Both models use class weighting. The tested Logistic Regression grid was C = 0.1, 1, 10; Random Forest grid searched 100/200 trees, unlimited/depth-6 trees and leaf sizes 1/2.



Selected model: holdout confusion matrix and ROC curve

Full grid results and selected parameters: `artifacts/evaluation/` and `artifacts/training_summary.json`. No external validation cohort, calibration analysis or subgroup fairness results are claimed.

MLflow runs and recorded outputs

Training creates the local MLflow experiment `heart_disease_prediction`. A parent run records the selected model; nested runs compare Logistic Regression and Random Forest. Parameters, CV metric means and standard deviations, holdout metrics, model artifacts, search tables and diagnostic plots are logged per model.

Tracked item	Recorded evidence	Repository artifact
Parameters	Model type, split seed/size, fold count, best grid parameters, selection metric	<code>mlruns/</code>
Metrics	CV mean and standard deviation; holdout accuracy, precision, recall, F1 and ROC-AUC	<code>artifacts/training_summary.json</code>
Models	Fitted preprocessing + estimator pipelines	<code>artifacts/best_model.joblib</code> , per-model joblib files
Plots and tables	Grid-search rows, confusion matrix and ROC curve	<code>artifacts/evaluation/</code>

Model registry / local UI

The local tracking URI points to the repository's `mlruns/` store. Inspect runs with:

```
$env:MLFLOW_ALLOW_FILE_STORE = "true"
mlflow ui --backend-store-uri .\mlruns --host 127.0.0.1 --port 5000
```

The Docker image includes the saved final model and loads it at service startup. If that artifact is absent during a development run, the API can train a replacement from the local dataset. For production, pin and version the model together with a dependency lock, dataset checksum and MLflow run ID.

Reproducibility boundary: fixed seeds and checked-in data/artifacts support repeatable local evaluation. Requirements are lower bounds, not a lockfile; platform/library updates may change exact fitted bytes and scores.

Validated FastAPI prediction API

Route	Purpose	Verification
GET /health	Process and model readiness	HTTP 200; model_loaded=true
POST /predict	13-feature validated patient payload	Prediction, class-1 probability and predicted-class confidence
GET /metrics	Prometheus counters and latency histogram	HTTP 200; request metrics observed
GET /docs	Interactive OpenAPI / Swagger documentation	Live browser evidence on page 8

Example request

```
POST /predict
Content-Type: application/json

{"age":52,"sex":1,"cp":0,"trestbps":125,"chol":212,
"fbs":0,"restecg":1,"thalach":168,"exang":0,"oldpeak":1.0,
"slope":2,"ca":2,"thal":2}
```

The response shape is `{"prediction":1,"probability":0.7242,"confidence":0.7242}` for this test request in the built container. The response rounds probability fields to four decimals. Pydantic constrains coded categories and rejects negative values; requests do not include raw feature values in logs.

Local access after Docker start

```
docker run --name heart-disease-api-local -p 8000:8000 heart-disease-api:latest
curl http://127.0.0.1:8000/health
curl http://127.0.0.1:8000/docs
```

The screenshot evidence folder contains a captured Swagger page, readiness JSON and Prometheus metrics response from the running Docker service. The live service is currently published on localhost port 8000.

The predicted label and score are model outputs only. A probability from this model is not a clinically calibrated individual risk estimate.

Checkout → Python 3.12 + dependencies → Ruff
→ pytest → training + MLflow → EDA → workflow artifacts

Check	Result	Scope
<code>python -m pytest -q</code>	PASS · 5 passed	API health, prediction, metrics and data/preprocessor tests
<code>python -m ruff check --select E4,E7,E9,F src api tests</code>	PASS · All checks passed	Source, API and tests
Docker image build	PASS · image built	heart-disease-api:latest
Docker health / prediction / metrics	PASS · HTTP 200	Running container on localhost:8000
GitHub Actions run #3	PASS · all steps	Workflow run · download artifact

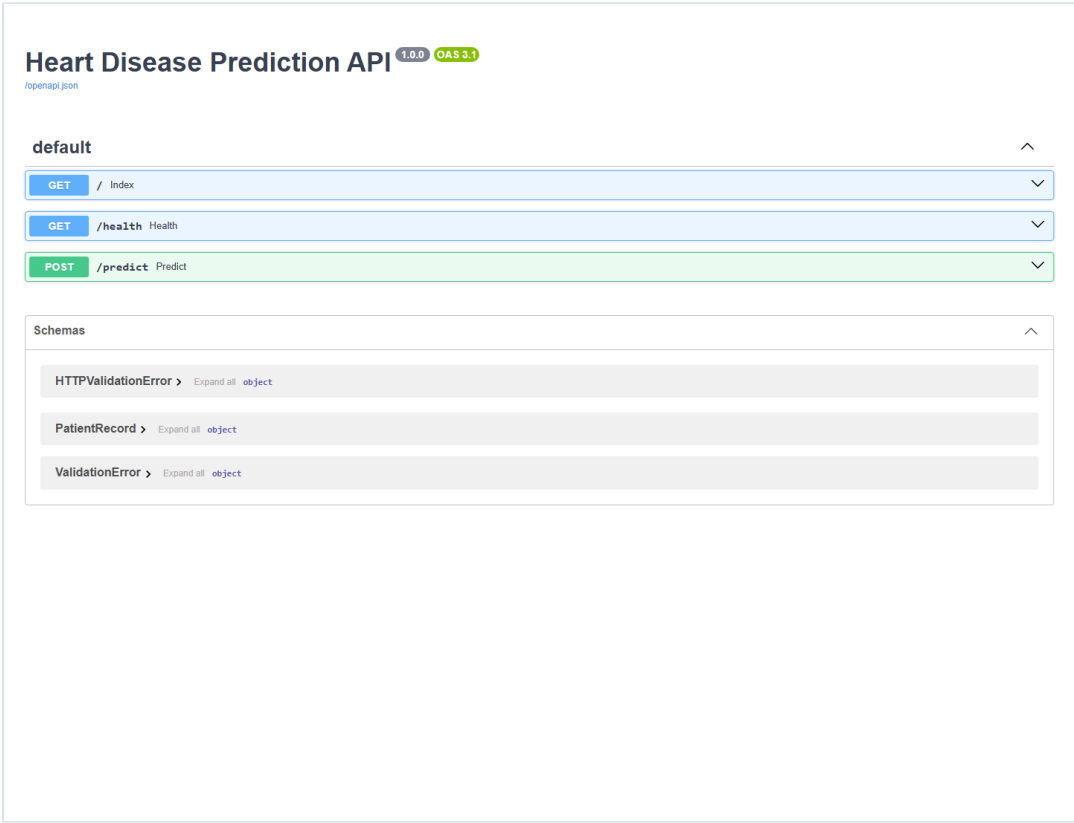


Reproduce the exact local quality gates from the project root: run the Ruff command above, followed by `python -m pytest -q`. Hosted run #3 uploaded the [heart-disease-mlops-artifacts](#) bundle (retained until 16 October 2026).

Docker verification evidence

The Docker image was built successfully from the repository's Python 3.12 slim base image using the declared requirements. Docker Desktop's Linux engine is running; the container remains up and is published on port 8000.

Build / runtime evidence	Observed result
Image tag	heart-disease-api:latest
Image ID	sha256:91ad6ca68e286581a95a3bb3aba233d26e6096ae1fb5db3c4aa3bd2f336eef65
Container name / status	heart-disease-api-local · Up, host 0.0.0.0:8000 → container 8000
/health	{"status":"ok","model_loaded":true}
Sample /predict	HTTP 200 · prediction 1 · probability 0.7242 · confidence 0.7242
/metrics	HTTP 200 · request counter and duration histogram exposed



Captured browser screenshot of the API running from the built Docker container; health and metrics screenshots are in [screenshots/](#).
Evidence date: 2 October 2026. Build context excludes the local virtual environment, Git data, MLflow run store and caches via `.dockerignore`.

Request observability and cluster status

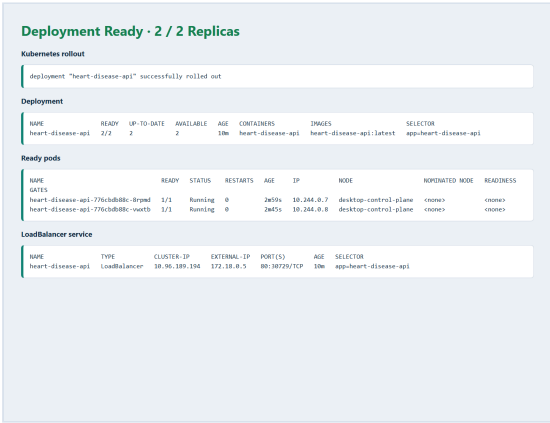
FastAPI middleware counts requests by method, route and status, observes request duration and emits structured access logs containing route, status and duration—not patient feature values. The Prometheus `/metrics` endpoint exposes counters and histograms. A Prometheus scrape configuration is provided in `monitoring/prometheus.yml`; Kubernetes manifests define a two-replica deployment, health probes, scrape annotations and a LoadBalancer service.

```
heart_disease_api_requests_total{method="GET",route="/health",status="200"} 1.0
heart_disease_api_requests_total{method="POST",route="/predict",status="200"} 1.0
heart_disease_api_requests_total{method="GET",route="/metrics",status="200"} 1.0
```

Deployment evidence boundary

Target	Status	Evidence / blocker
Local Docker API	DEPLOYED & tested	Container running; health, predict, metrics verified; screenshots saved
Docker Desktop Kubernetes	DEPLOYED · 2/2 READY	Successful rollout; health and sample prediction verified through port-forward
Public cloud URL	NOT CLAIMED	No cloud credentials or public host configured
Prometheus server / Grafana	CONFIG PROVIDED	API metrics endpoint works; no separate Prometheus/Grafana server was launched

The Docker Desktop Kubernetes rollout reached two ready replicas. Its containerd image store is separate from Docker's local image store, so the built image was explicitly imported and the manifest uses `imagePullPolicy: Never`. The service was tested through `kubect1 port-forward service/heart-disease-api 8080:80`. No public deployment URL is claimed. The scrape target in the Prometheus config is cluster DNS and must be changed for an external local Prometheus instance.



Docker Desktop Kubernetes deployment and two ready pods. Additional Kubernetes API screenshots are in [screenshots/](#).

Hosted CI evidence: the previous hosted run exposed a pytest import-path error. The import path is now fixed in `pytest.ini`; the corrected workflow run is pending.

Coverage summary and next actions

Assignment area	Coverage	Repository artifact / evidence
1. Data acquisition & EDA	Implemented	Download-on-missing, data snapshot, histograms, heatmap, balance and summary
2. Feature engineering & modeling	Implemented	Leakage-aware preprocessing, two grid-searched models, five-fold CV and metrics
3. Experiment tracking	Implemented	Nested MLflow runs with parameters, metrics, artifacts and plots
4. Packaging & reproducibility	Implemented	Serialized full pipeline, requirements, inference command
5. CI/CD & testing	Workflow fixed / pending hosted rerun	GitHub Actions, Ruff, 5 pytest tests; <code>pytest.ini</code> fixes CI imports
6. Model containerization	Built, started & verified	Dockerfile, live API and captured screenshots
7. Production deployment	Local Kubernetes deployed	Two ready replicas; Service and API verified; no public cloud URL
8. Monitoring & logging	API endpoint verified	Request logs and Prometheus metrics; external dashboard not launched
9. Documentation & reporting	PDF + Markdown	This 10-page PDF, <code>report.md</code> , README, EDA and API evidence

Final external deliverables still requiring your environment

1. Record a short video walkthrough of the pipeline using your own screen recorder; this workspace cannot create an authentic user-screen recording.
2. Run the corrected GitHub Actions workflow and include its run URL/artifact if hosted CI proof is required.
3. Optionally deploy to a cloud provider and add its reachable API URL; no public endpoint is claimed.

Limitations: historical small dataset, no external validation, no calibration/fairness/drift assessment, no clinical review, no Grafana dashboard, and no video capture. This model must not be used for real medical decisions.

Key deliverables: source in `src/` and `api/`; data and EDA/model artifacts under `data/` and `artifacts/`; tests in `tests/` with `pytest.ini`; CI at `.github/workflows/ci.yml`; Dockerfile, `k8s/deployment.yaml` and `scripts/load_docker_desktop_image.ps1`; evidence in `screenshots/`.